

PRACTICE I - FROM PIXELS TO THE INTEGRAL: AREA UNDER A CURVE

Functional programming with Haskell · Logic programming with Prolog

Item	Detail
Course	Programming Paradigms
Lecturer	Alexander Narváez — anarvae1@eafit.edu.co
Value	12% of the final grade of the course
Team	Same teams as Practice I
Submission	GitHub repository + video presentation
Input file	curva_binaria_P4.pbm (supplied by the lecturer)

What you must deliver

Deliverable	Description
Haskell program	Reads the PBM P4 file, builds M, computes the area, draws the curve in the console
Prolog program	Same result, expressed as relations rather than as a procedure
GitHub repository	README.md, Haskell/, Prolog/, curva_binaria_P4.pbm
Video presentation	Both programs running, plus the defence of the code

Both programs must obtain the same area from the same file. That number is the check that the two paradigms agree.

INSTRUCTIONS

The purpose of this practice is to explore how the **same mathematical problem can be expressed using two different programming paradigms: Functional Programming with Haskell and Logic Programming with Prolog**.

The lecturer will provide a binary image in **PBM P4 format** representing a region under a curve. The objective is to read the binary information, interpret each vertical column as the value of a discrete function $f(x)$, and calculate the area under the curve using a **Riemann sum**.

The original PBM image is considerably larger than the dimensions normally available in a terminal window. Therefore, students must design an appropriate strategy to produce a **scaled or compact visual representation in the console**, while preserving the general shape of the original curve.

The lecturer will provide the file `curva_binaria_P4.pbm`, which must be used directly as the data source for the exercise.

MATHEMATICAL INTERPRETATION

For every horizontal position x , the program must inspect the corresponding column of the image from the bottom upward and count the consecutive black pixels. This number represents the height of the function:

$$f(x) = \text{number of consecutive black pixels from the bottom of column } x$$

Therefore, the complete image can be transformed into a vector or list of heights:

$$M[x] = f(x)$$

or, equivalently,

$$M = [f(0), f(1), f(2), \dots, f(n-1)]$$

If each pixel column has width

$$\Delta x = 1 \text{ pixel,}$$

then each column can be interpreted as one rectangle of a Riemann sum:

$$A = \sum_{i=0}^{n-1} f(x_i) \Delta x$$

Since $\Delta x = 1$,

$$A = \sum_{i=0}^{n-1} f(x_i)$$

The result must be expressed in **square pixels**.

```

MakeLists.txt  main.cpp  integral.cpp x
;7 //Autor: Alexander Narváez - anarvae1@eafit.edu.co
;8 // CARGAR PBM P4
;9 // =====
;0 ifstream archivo( S: "../curva_binaria_P4.pbm", mode: ios::binary);
;1 if (!archivo) {

main

integral x
C:\Users\Windows\CLionProjects\calc\cmake-build-debug\integral.exe
imagen: 567 x 319 pixeles
Area = 108660 pixeles cuadrados

VECTOR DE ALTURAS M[x] = f(x)

ALGUNOS VALORES x_i -> f(x_i)
x_0 = 0 -> f(x_0) = 224 pixeles
x_1 = 62 -> f(x_1) = 239 pixeles
x_2 = 125 -> f(x_2) = 227 pixeles
x_3 = 188 -> f(x_3) = 225 pixeles
x_4 = 251 -> f(x_4) = 237 pixeles
x_5 = 314 -> f(x_5) = 229 pixeles
x_6 = 377 -> f(x_6) = 176 pixeles
x_7 = 440 -> f(x_7) = 113 pixeles
x_8 = 503 -> f(x_8) = 97 pixeles
x_9 = 566 -> f(x_9) = 145 pixeles

Cada columna tiene base = 1 pixel
Area = suma de f(x_i)
Area = 108660 pixeles cuadrados
Process finished with exit code 0

```

Figure 1. Finding the area under the curve. Application developed in C++ by Alexander Narváez.

Figure 1 shows an example of the type of output that can be obtained when the reference C++ program is executed. The objective is **not to translate the C++ implementation line by line**, but to obtain an equivalent result using the principles of Haskell and Prolog.

The console representation does not have to display every pixel of the original file. Since the original image is much larger than the console, students must determine how to scale, sample, compress or otherwise represent it while maintaining a recognizable approximation of the original curve.

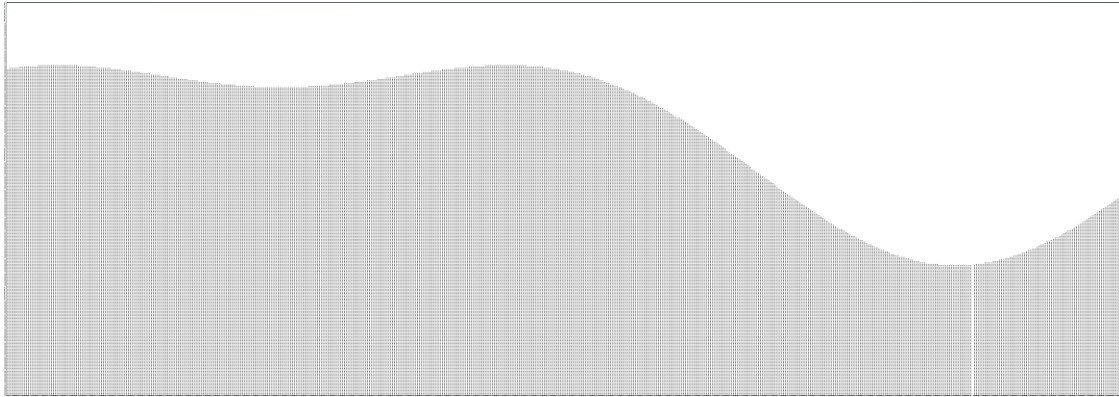


Figure 2. Approximate view of the original PBM P4 binary image supplied as the input source. Because of its dimensions, its console representation requires a scaling or compacting strategy.

FUNCTIONAL REQUIREMENTS

- 1. Load the PBM P4 file.** Open and correctly interpret the binary PBM file supplied by the lecturer. Remember that PBM P4 is a binary format and its pixels are compacted into bits; one byte may contain information for up to eight pixels.
- 2. Access individual pixels.** Implement the necessary mechanism to determine whether the pixel located at coordinates (x, y) is black or white. The solution must demonstrate understanding of the bit representation used by the PBM P4 format.
- 3. Construct the discrete function $f(x)$.** For each horizontal position x , inspect the corresponding column starting at the bottom of the image. Count the number of consecutive black pixels until the first white pixel is encountered. The resulting value is $f(x)$.
- 4. Construct the height structure.** Generate a data structure containing the heights of all columns:

$$M = [f(0), f(1), f(2), \dots, f(n-1)].$$

- 5. Calculate the area using a Riemann sum.** Since every column has base $\Delta x = 1$ pixel, calculate

$$A = \sum_{x=0}^{n-1} f(x).$$

Display the resulting area in **square pixels**.

- 6. Visualize the binary image in the console.** Produce a console representation similar in spirit to Figure 1. Because the source image is much larger than a terminal window, the team must design and justify a strategy such as spatial sampling, horizontal or vertical scaling, grouping several pixels into one terminal character, Unicode block characters, or another suitable method. The resulting representation should preserve the **general shape of the original curve**.
- 7. Visualize the height function.** Display a graphical representation of $M[x] = f(x)$ in the console. Unicode blocks or another text-based visualization may be used to make variations in height visible.
- 8. Show sample values.** Display at least **10 positions** distributed throughout the domain. For each selected position, show

$$x_i \rightarrow f(x_i).$$

For example:

`x_0 = ... -> f(x_0) = ... pixels`

`x_1 = ... -> f(x_1) = ... pixels`

...

PART I - HASKELL

The complete problem must first be implemented in **Haskell**.

The objective is not simply to reproduce imperative loops using recursion. The solution should reflect the principles of the **functional programming paradigm**.

A fundamental idea is that the height structure is the application of one function to every value of the domain:

$$M = [f(0), f(1), f(2), \dots, f(n - 1)].$$

In functional terms, this can naturally be expressed conceptually as:

`m = map f [0 .. ancho - 1]`

Once the list of heights has been produced, the area should emerge naturally from:

`area = sum m`

Conceptually, the Haskell solution should make this transformation visible:

$$\text{PBM} \rightarrow \text{bytes} \rightarrow \text{pixels} \rightarrow f(x) \rightarrow M \rightarrow \text{area}$$

Here, the construction of M should emphasize the functional transformation of the domain through f , for example with `map`, and the area should be obtained from the resulting structure, for example with `sum`.

Haskell technical requirements

- The solution must be developed in Haskell.
- File handling must be implemented.
- Binary information from the PBM P4 file must be processed correctly.
- Appropriate functions must be defined for the different stages of the solution.
- The height structure must be represented using an appropriate Haskell data structure.
- Functional transformations such as `map`, list comprehensions, recursion, folds or equivalent techniques should be used appropriately.
- The mathematical relationship between $f(x)$, the list of heights and the final sum must be identifiable in the implementation.
- The program must produce a console visualization of the source curve and the height function.

PART II - PROLOG

The same problem must also be solved in **Prolog**.

However, the purpose is not to translate the Haskell or C++ solution instruction by instruction. The Prolog implementation should reformulate the problem from a **logic/declarative perspective**.

Instead of asking *What sequence of instructions must be executed?*, the solution should emphasize *What relationships must hold between the image, a position x , its height $f(x)$ and the resulting area?*

For example, a relation of the form

`f(X, Datos, Alto, BytesPorFila, Altura)`

can express that *Altura* is the height associated with column *X* in the supplied binary image.

Once this relation has been defined, Prolog can obtain the list of heights declaratively. A conceptual structure could be:

```
findall(Altura,
  (between(0, MaxX, X),
    f(X, Datos, Alto, BytesPorFila, Altura)),
  M).
```

The complete area can then be obtained from that list, for example:

`sum_list(M, Area).`

Conceptually:

relations $\rightarrow$ values that satisfy $f(x) \rightarrow M \rightarrow \text{area}$

Prolog technical requirements

- The complete problem must be implemented in Prolog.
- The PBM P4 file must be read and interpreted directly.
- Relations or predicates must be defined to represent access to pixels and the calculation of $f(x)$.
- The list of heights must be constructed declaratively.
- The use of mechanisms such as `findall/3`, recursion, arithmetic relations and list processing must be justified where appropriate.
- The solution should demonstrate the logic-programming paradigm rather than merely simulate an imperative algorithm.
- The program must produce a console visualization of the source curve and the height function.

COMPARISON OF PARADIGMS

As part of the practice, students must explain how the same mathematical expression

$$A = \sum_{i=0}^{n-1} f(x_i) \Delta x$$

is represented differently in Haskell and Prolog.

Haskell perspective

The problem can be viewed as a sequence of transformations:

$$\text{domain} \rightarrow \text{heights} \rightarrow \text{area}$$

The first transformation corresponds to applying f to the domain, while the second corresponds to accumulating the resulting heights. The emphasis is on **functions and transformations of data**.

Prolog perspective

The problem can be viewed as a collection of logical relationships:

$$f(X, Y) \rightarrow \text{find all valid } Y \rightarrow \text{area}$$

The emphasis is on **describing which relationships must hold and asking Prolog to find the values that satisfy them**.

Students must be able to explain this difference during the presentation and defense.

TECHNICAL REQUIREMENTS

- The practice must include **two complete implementations**: Haskell and Prolog.
- Both programs must use the PBM P4 file supplied by the lecturer.
- The PBM file must **not** be manually converted into a text matrix before execution.
- Both implementations must calculate the same area for the supplied image.
- Both implementations must produce a visual console representation of the image or curve.
- Because the source image is significantly larger than a terminal window, students must implement and explain their strategy for reducing its visual representation.
- The mathematical function $f(x)$ must be clearly identifiable in both solutions.
- The resulting list or vector of heights must be clearly identifiable.
- The Riemann sum must be explicitly related to the calculated area.

SUPPORTING MATERIAL

The lecturer will provide:

- The original **PBM P4 binary image** (curva_binaria_P4.pbm).
- The reference images shown in this document.
- An example of the result obtained with a C++ implementation.
- Additional explanatory material if required.

The C++ reference is intended to explain the expected behavior of the application. **It must not be submitted as the solution to the practice.**

GITHUB REPOSITORY

Please share the repository through GitHub. The repository must contain at least:

README.md
Haskell/
Prolog/
curva_binaria_P4.pbm

The README.md file must include:

- Names of all team members.
- Development environment or platform used.
- Instructions to compile or execute the Haskell solution.
- Instructions to execute the Prolog solution.
- A short explanation of the strategy used to display the large image in the console.
- The area obtained from the supplied PBM file.

PRESENTATION AND DEFENSE

Finally, make a short video presentation explaining and demonstrating how the practice works on your own computer.

1. Show the original PBM P4 image.
2. Execute the Haskell solution.
3. Show the console visualization and calculated area in Haskell.
4. Show some values $x_i \rightarrow f(x_i)$.
5. Execute the Prolog solution.
6. Show the console visualization and calculated area in Prolog.
7. Explain why the Haskell solution is functional.
8. Explain why the Prolog solution is declarative/logical.
9. Compare both approaches.
10. Explain the connection between the program and the Riemann sum.

Students must be prepared to **justify the code**, not only demonstrate that it executes correctly.

CONCEPTUAL OBJECTIVE

At the end of the practice, the student should be able to explain the complete transformation:

$$\text{binary image} \rightarrow \text{bits} \rightarrow \text{pixels} \rightarrow f(x) \rightarrow M[x] \rightarrow \sum f(x_i)\Delta x \rightarrow \text{area}$$

and recognize that although the mathematical problem remains the same, its computational expression changes according to the programming paradigm being used.

ASSESSMENT

Following the same evaluation structure used in the previous practice:

- **Program operation in Haskell and Prolog (30%)**
- **Presentation and defense (70%)**